

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INGEGNERIA INDUSTRIALE
Corso di Laurea Magistrale in Ingegneria Energetica

Relazione di Progetto:

**DIGITALIZATION OF ENGINEERING METHODOLOGIES AND
DATA**

Realizzato da:

Galletta Roberta
Madrigali Andrea
Perilli Pietro
Poli Edoardo

Professore:

Ghedini Emanuele

Anno Accademico 2025/2026

Sommario

Introduzione:	3
Cap1: Dati	4
Cap2: Fase Preliminare Mirò	6
2.1 Introduzione a TBox ed ABox:	6
2.2 TBox	7
2.3 ABox	8
2.4 Mirò:	9
Cap3: Approccio su Protegé.....	13
Cap4: Reference su GraphDB e querying	16
Conclusioni:	23
Bibliografia:	24

Introduzione:

L'obiettivo di questo lavoro è stato quello di organizzare e rendere interrogabili i dati di simulazione relativi al reattore AP600, superando i limiti di una rappresentazione puramente tabellare. I dati analizzati provengono da simulazioni eseguite tramite il software certificato Passive PWR, ampiamente utilizzato per lo studio del comportamento transitorio dei reattori ad acqua pressurizzata.

Il focus dell'ontologia non è quindi la descrizione del reattore reale in senso fisico-ingegneristico, bensì la strutturazione semantica dei dati prodotti dalle simulazioni, con particolare attenzione alle grandezze fisiche osservate ed agli eventi discreti di interesse (quale lo SCRAM, il turbine trip o la rottura della cold leg). I dati iniziali, sebbene completi dal punto di vista numerico, risultano infatti poco espressivi sul piano semantico e difficili da interrogare in modo flessibile. Per questo motivo si è scelto di passare ad una modellazione ontologica, in grado di esplicitare le relazioni tra componenti, grandezze e stati del sistema simulato.

L'ontologia è stata progettata seguendo un approccio incrementale: partendo da una fase concettuale preliminare, per poi arrivare ad una formalizzazione in OWL, utilizzata secondo un approccio leggero di tipo RDFS+, tramite Protégé, ed al successivo utilizzo in GraphDB per il querying.

La scelta di rappresentare i dati temporali come stringhe vettoriali direttamente associate agli individui consente di evitare una fase di mapping esterna, semplificando il passaggio verso il database semantico e mantenendo il focus sugli aspetti concettuali richiesti dal corso.

Cap1: Dati

Per quanto concerne la raccolta dei dati, sono state eseguite due simulazioni differenti utilizzando il software certificato **Passive PWR**. Nella prima simulazione è stato impostato un valore di *target output power* pari al 100%, mentre nella seconda pari al 68%. In entrambe le simulazioni sono stati mantenuti invariati il valore del *Power Rate Change* (%/s), la modalità operativa del reattore (*Reactor Lead*) e la tipologia di malfunzionamento introdotto, ovvero **RC Cold Leg #4 – LOCA BREAK**, corrispondente alla rottura della cold leg del loop 4 del circuito primario. Tale evento determina una perdita di refrigerante, una rapida depressurizzazione del sistema e la conseguente attivazione dell'arresto automatico del reattore (SCRAM) e dei sistemi di sicurezza.

Una volta avviata la simulazione, per gli istanti temporali

$$t = \{0, 10, 20, 30, 40, 65, 185, 385\} \text{s}$$

sono stati raccolti i valori dei principali parametri forniti dal software (tra cui *Neutron Power [%]*, *Thermal Power [%]*, *Generator Output [%]*, pressione, portata di nocciolo, ecc.). In una fase iniziale, tali dati sono stati salvati e organizzati tramite tabelle **Excel**, che hanno rappresentato il primo strumento di archiviazione e visualizzazione numerica dei risultati.

Le simulazioni effettuate consentono di ottenere un insieme completo e accurato di dati numerici relativi all'evoluzione temporale dei principali parametri del reattore. Tuttavia, questi risultati, pur essendo corretti e significativi dal punto di vista fisico-ingegneristico, risultano intrinsecamente limitati sotto il profilo semantico.

I dati sono infatti organizzati prevalentemente come serie temporali o tabelle numeriche, prive di un'esplicita descrizione del significato concettuale delle grandezze, delle relazioni tra i componenti del sistema e degli eventi che caratterizzano il transitorio. In questo contesto, l'obiettivo del lavoro è la strutturazione semantica delle informazioni prodotte dalle simulazioni, con particolare attenzione sia alle grandezze fisiche osservate sia agli eventi di interesse dal punto di vista operativo e di sicurezza, quali lo SCRAM, la *turbine trip* o la rottura della cold leg. Per perseguire tale obiettivo, si è scelto di adottare un approccio basato sulla modellazione ontologica, sviluppata tramite il software **Protégé**. L'ontologia consente di esplicitare formalmente i concetti rilevanti del dominio simulato (componenti, grandezze, stati ed eventi), nonché le relazioni semantiche che intercorrono tra di essi.

La rappresentazione dei dati mediante tabelle consente di organizzarli secondo una struttura bidimensionale, in cui alle righe corrispondono gli istanti temporali e alle colonne i valori numerici delle grandezze considerate. Sebbene questa modalità risulti efficace per la memorizzazione e la consultazione dei dati, le informazioni rimangono "piatte": la tabella non permette infatti di codificare in modo esplicito il tipo di evento, il componente che lo genera, le grandezze che esso influenza o lo stato del sistema in cui avviene. Tutti questi aspetti restano impliciti e demandati all'interpretazione dell'utente.

Attraverso la modellazione ontologica è invece possibile passare da una struttura bidimensionale a una rappresentazione concettuale multidimensionale. In questo modo un evento non è più

semplicemente un valore numerico associato a una cella della tabella, ma diventa un oggetto semantico dotato di attributi e relazioni, in grado di fornire informazioni sulla sua natura tipologica (che tipo di evento è), strutturale (quale componente del sistema coinvolge), causale (quali condizioni o eventi lo hanno generato) e funzionale (quali grandezze del sistema influenza).

A titolo di esempio, si riporta di seguito l'interfaccia utente del software **Passive PWR** dalla quale sono stati estratti i dati di simulazione.

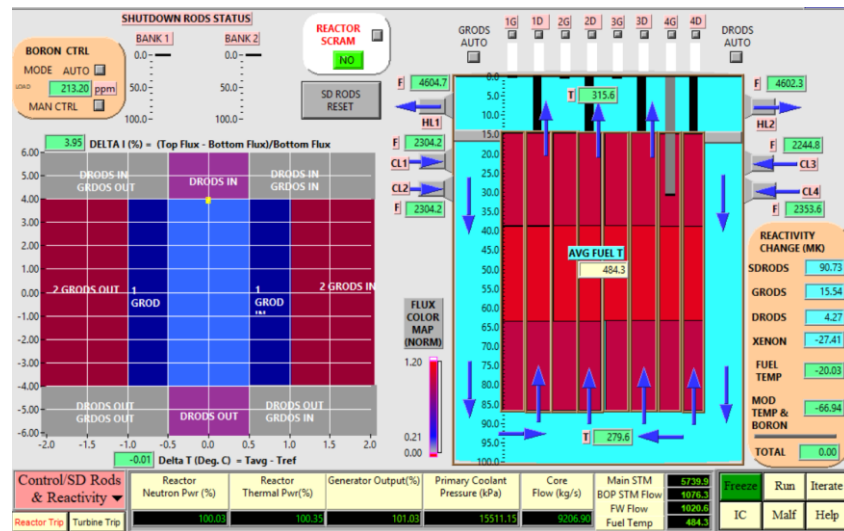


Figura 1: Passive_PWR Interfaccia Utente

L'obiettivo finale del lavoro è quindi quello di passare da un semplice salvataggio di dati "raw" a una rappresentazione ontologica semanticamente corretta e funzionante, in grado di supportare operazioni di querying e di verifica della coerenza tra i dati, valorizzando in modo strutturato le informazioni prodotte dalle simulazioni.

Cap2: Fase Preliminare Mirò

2.1 Introduzione a TBox ed ABox:

Nel momento in cui si costruisce una knowledge base in Description Logics, cioè l'insieme strutturato di informazioni che il tool utilizza come fondamento per eseguire il reasoning, diventa possibile interrogare il sistema secondo il metodo Tell & Ask. In questa fase emerge una distinzione fondamentale fra due tipi di conoscenza: la *intensional knowledge*, detta anche *general knowledge*, che descrive la struttura concettuale generale del dominio modellato, e la *extensional knowledge*, che riguarda invece informazioni specifiche relative alle istanze concrete considerate nel problema. La knowledge base è in genere composta quindi da due componenti: la TBox e la ABox. La TBox contiene la conoscenza *intensional* nella forma di terminologia del dominio (spesso descritta anche come tassonomia, da cui deriva la 'T'), ed è costruita mediante assiomi che esprimono proprietà generali dei concetti e delle relazioni tra essi (in particolare, poiché le relazioni fra concetti sono spesso modellate tramite *subsumption relationships*, ovvero inclusioni fra classi, la TBox presenta tipicamente una struttura gerarchica). La ABox, invece, contiene la *extensional knowledge*, detta anche *assertional knowledge* (da cui deriva il termine ABox), la quale rappresenta la parte "fattuale" della knowledge base e descrive gli individui del dominio, specificandone l'appartenenza a determinati concetti e le relazioni che li collegano. In altre parole, mentre la TBox definisce il vocabolario concettuale e le regole generali del dominio, la ABox fornisce i dati concreti su cui tali regole vengono applicate. Questa distinzione risulta essenziale per comprendere la costruzione delle mappe concettuali adottate nel lavoro e per interpretare correttamente sia la gerarchia delle classi, sia la separazione tra livello terminologico (TBox) e livello istanziale (ABox).

Chiarimento su intensional vs extensional knowledge

La **intensional knowledge** (TBox) è chiamata così perché descrive il "significato" dei concetti del dominio: definisce classi, proprietà e vincoli generali, cioè lo schema concettuale. Per questo motivo è considerata generalmente stabile: non cambia ogni volta che cambiano i dati, perché descrive la struttura del dominio, non la situazione particolare.

La **extensional knowledge** (ABox), invece, riguarda le istanze, ovvero gli individui definiti ed i fatti che li riguardano. È considerata più dinamica, perché può cambiare frequentemente a seconda dello scenario, dei dati osservati o della situazione concreta che si vuole descrivere. Se ad esempio si aggiunge un nuovo individuo o si verifica la modifica di una relazione esistente tra individui, si ha l'aggiornamento dell'ABox (e non della TBox), quindi si può affermare che è specifica per una particolare situazione o un particolare insieme di dati all'interno del dominio stesso.

2.2 TBox

Un elemento chiave di una knowledge base è rappresentato dalle operazioni necessarie alla costruzione della terminologia del dominio, le quali sono strettamente legate alla forma e al significato delle dichiarazioni ammesse all'interno della TBox. La TBox è fondamentale perché rappresenta la componente della knowledge base in cui vengono definiti i concetti del dominio, cioè le classi utilizzate per descrivere e organizzare gli individui. Una classe può essere intesa come l'insieme (o categoria) di tutti gli individui che soddisfano determinate condizioni: tali condizioni sono espresse tramite proprietà e vincoli logici (ad esempio relazioni, restrizioni su attributi o appartenenza a sottoclassi). In questo modo, la TBox fornisce la struttura concettuale generale del dominio e stabilisce i criteri secondo cui gli individui verranno successivamente classificati nella ABox.

Nella pratica, la terminologia DL viene spesso costruita assumendo alcune condizioni comuni nelle knowledge base, tra le quali si hanno che:

- A ciascun nome di concetto viene associata al più una definizione, ossia non sono consentite definizioni multiple per lo stesso concept name;
- Le definizioni sono tipicamente acicliche: non è possibile che un concetto compaia (direttamente o tramite una catena di rimandi) nella propria definizione. Per esempio, non è ammesso definire A usando A stesso ($A \equiv \dots A \dots$), ma nemmeno definire A tramite un concetto B che a sua volta sia definito tramite A ($A \equiv \dots B \dots$ e $B \equiv \dots A \dots$), perché ciò creerebbe un ciclo.

Queste restrizioni implicano una conseguenza importante: ogni concetto definito può essere “sviluppato” in modo univoco fino a ottenere un'espressione complessa che contiene soltanto concetti atomici, cioè concetti primitivi non ulteriormente definiti. Di conseguenza, attraverso l'espansione delle definizioni si ottiene un'espressione formulata esclusivamente mediante concetti atomici, ovvero quelli che non vengono ulteriormente definiti all'interno della terminologia (tali concetti costituiscono gli elementi primitivi del vocabolario concettuale, a partire dai quali sono costruite tutte le definizioni più complesse).

Tale espansione si realizza sostituendo progressivamente ogni concetto definito con l'espressione che compare sul lato destro (right-hand side) della sua definizione: in pratica una classe può essere introdotta tramite un assioma di definizione, e il nome della classe funziona quindi come un'etichetta che abbrevia un'espressione concettuale più articolata. Ciò che compare sul lato destro della definizione descrive le condizioni necessarie (e nel caso di equivalenza anche sufficienti) per appartenere a quella classe, di conseguenza, durante l'espansione (unfolding) è possibile sostituire il nome della classe con l'intera espressione definitoria presente a destra, ottenendo una formulazione equivalente ma priva del concetto definito come simbolo “compresso”.

Gran parte dello studio dei metodi di reasoning nelle Description Logics si concentra sulle espressioni concettuali e, in particolare, sulla relazione di subsumption: la subsunzione permette di

stabilire se un concetto è più generale di un altro e costituisce la base per la costruzione e la validazione della gerarchia concettuale. Nel contesto di Protégé questa operazione è cruciale perché consente al reasoner di classificare automaticamente l'ontologia: a partire dagli assiomi definiti nella TBox, il sistema determina le relazioni di inclusione tra classi, organizza correttamente la tassonomia e verifica la coerenza della struttura concettuale. In questo modo, la definizione delle classi e dei vincoli terminologici non ha soltanto un ruolo descrittivo, ma diventa il presupposto operativo che permette al reasoner di rispondere alle interrogazioni e di supportare il metodo Tell & Ask.

2.3 ABox

La ABox rappresenta la componente della knowledge base dedicata alla conoscenza estensionale del dominio, ossia l'insieme dei fatti relativi agli individui (in essa vengono infatti inserite asserzioni che descrivono oggetti specifici del dominio e le relazioni che intercorrono tra essi). Per questo motivo, le affermazioni contenute nella ABox vengono generalmente indicate come membership assertions, poiché stabiliscono l'appartenenza di un individuo a un determinato concetto.

Un primo tipo di asserzione è costituito dalle concept assertions (asserzioni concettuali), che indicano che un individuo appartiene a una classe: ad esempio, l'espressione $\text{Female} \sqcap \text{Person}(\text{ANNA})$ afferma che l'individuo ANNA è una persona di sesso femminile. Se nella TBox è stata precedentemente definita la classe Woman come equivalente all'intersezione $\text{Person} \sqcap \text{Female}$, allora a partire da tale asserzione il reasoner è in grado di dedurre automaticamente che ANNA è un'istanza del concetto Woman (la ABox fornisce i dati concreti, mentre la TBox fornisce le definizioni concettuali che permettono di derivare nuova conoscenza). Un secondo tipo di asserzione è costituito dalle role assertions (asserzioni di ruolo), che specificano relazioni tra individui (per esempio, $\text{hasChild}(\text{ANNA}, \text{JACOPO})$ indica che ANNA ha come figlio JACOPO).

Il valore della ABox emerge pienamente quando interviene il reasoner: le asserzioni sugli individui, lette alla luce della TBox, permettono infatti di dedurre appartenenze a classi in modo implicito. Di conseguenza, un individuo può risultare istanza di un concetto anche senza che tale appartenenza sia stata dichiarata esplicitamente, ma perché logicamente derivabile dagli assiomi dell'ontologia: in questo contesto, il principale compito di reasoning nella ABox è l'instance checking, ovvero la verifica che un individuo sia o meno istanza di un certo concetto. Questo tipo di controllo è centrale perché consente di rispondere a interrogazioni del tipo "Questo individuo appartiene a questa classe?". Tra le varie modalità di checking si trovano: la consistenza della knowledge base, che verifica se la knowledge base ammette un modello (ovvero se non contiene contraddizioni e se i concetti possono avere istanze); la realization, che determina il concetto più specifico a cui un individuo appartiene; e il retrieval, che individua tutti gli individui che sono istanze di un concetto dato. Tali operazioni permettono, in pratica, di estrarre informazione utile dalla base di conoscenza e sono essenziali nelle interrogazioni e nella classificazione automatica degli individui.

La presenza di individui nella knowledge base rende il reasoning più complesso dal punto di vista computazionale rispetto al caso in cui si ragiona soltanto sulla TBox perché, quando presenti individui, il reasoner non deve più limitarsi a determinare relazioni tra concetti (ad esempio verificare subsunzioni e coerenza della tassonomia), ma deve anche stabilire quali concetti descrivano correttamente ciascun individuo e quali relazioni tra individui siano compatibili con i vincoli definiti nella terminologia. Inoltre, quando la TBox è aciclica e definizionale, molte inferenze possono essere semplificate “espandendo” i concetti definiti tramite sostituzione delle definizioni (unfolding). Tuttavia, se la TBox è più generale e non può essere gestita tramite questo semplice meccanismo, allora il reasoning deve necessariamente considerare congiuntamente entrambe le componenti della knowledge base: da un lato gli assiomi terminologici della TBox, dall’altro i fatti sugli individui presenti nella ABox. In queste condizioni il ragionamento diventa più articolato, perché le conclusioni su individui e concetti dipendono dall’interazione tra definizioni generali e dati specifici.

2.4 Mirò:

Prima della costruzione dell’ontologia in Protégé è stata svolta, mediante la realizzazione di mappe concettuali tramite Mirò, una fase preliminare di analisi e preparazione delle informazioni poi da inserire nella ontologia: questo passaggio ha avuto l’obiettivo di strutturare in modo chiaro e condiviso la conoscenza del dominio, identificando i concetti rilevanti, le relazioni principali tra essi e i vincoli logici implicati dal problema (Miro presenta lo scopo di fornire una visione d’insieme della struttura ontologica e di facilitare la progettazione del modello). In particolare, le mappe concettuali hanno permesso di definire un quadro iniziale coerente, riducendo il rischio di introdurre classi ridondanti o relazioni ambigue durante l’implementazione formale dell’ontologia.

Dal punto di vista metodologico, la costruzione delle mappe ha rappresentato una traduzione preliminare e informale della componente terminologica della knowledge base: attraverso la mappa concettuale vengono infatti individuate le classi e la loro organizzazione gerarchica, che corrispondono alla futura struttura della TBox. Allo stesso tempo, questa fase ha favorito l’individuazione delle proprietà e delle dipendenze tra concetti (relazioni del tipo “è un”, “parte di”, “possiede”, “dipende da”) che, successivamente, sono state formalizzate nell’ontologia tramite assiomi di inclusione, restrizioni e proprietà oggetto/dato.

Una volta definito lo schema concettuale in Miro, il modello è stato trasferito in Protégé: la gerarchia individuata tra i concetti è stata formalizzata come tassonomia di classi, mentre le relazioni e gli attributi emersi dalla mappa concettuale sono stati tradotti rispettivamente in object properties e data properties, così da preservare la struttura relazionale e descrittiva del dominio. Successivamente, in fase di popolamento della ABox, tali proprietà sono state istanziate sul dataset: le data properties sono state utilizzate per assegnare valori descrittivi agli individui, mentre le object properties hanno permesso di rappresentare esplicitamente le relazioni tra singoli individui (role assertions), creando i collegamenti necessari per il reasoning e per l’inferenza di appartenenze di classe implicite. Questa procedura ha consentito di mantenere coerenza tra rappresentazione

concettuale iniziale e formalizzazione logica, garantendo che l'ontologia riflettesse correttamente il dominio del problema e permettendo al reasoner di operare classificazione e inferenza in modo robusto.

In questa fase preliminare è risultato particolarmente utile avere chiara la distinzione tra TBox e ABox, poiché ha guidato fin dall'inizio l'organizzazione del lavoro. Attraverso le mappe concettuali è stata infatti definita la componente terminologica della knowledge base, identificando le classi rilevanti e la loro gerarchia (TBox), ma anche le proprietà necessarie a descrivere il dominio. In particolare, sono state individuate sia le object properties, impiegate per modellare relazioni tra concetti e quindi tra futuri individui, sia le data properties, utilizzate per associare agli individui attributi descrittivi espressi tramite valori letterali (ad esempio numeri, stringhe o date).

Per quanto riguarda le object properties, esse hanno avuto un ruolo centrale nella modellazione preliminare, poiché consentono di rappresentare formalmente le relazioni tra entità del dominio. A differenza delle data properties, che associano agli individui valori letterali (come numeri o stringhe), le object properties collegano un individuo a un altro individuo e permettono quindi di esprimere la struttura relazionale del dominio. Nella fase di progettazione su Miro sono state individuate le relazioni più rilevanti (ad esempio relazioni di dipendenza, appartenenza o composizione), che successivamente in Protégé sono state formalizzate come object properties, specificandone dominio e range e introducendo eventuali vincoli (come restrizioni esistenziali/universali o cardinalità).

Questo passaggio è risultato fondamentale perché tali proprietà costituiscono la base per inferenze automatiche: il reasoner può infatti dedurre nuove appartenenze di classe o verificare coerenza e compatibilità tra individui proprio sfruttando i vincoli definiti sulle object properties. Per quanto riguarda le data properties, esse sono state definite per associare agli individui attributi descrittivi espressi tramite valori letterali, quali stringhe, numeri o date. In questo senso, le data properties non modellano relazioni tra entità del dominio (come avviene per le object properties), ma permettono di rappresentare caratteristiche intrinseche degli individui, rendendo più precisa e informativa la descrizione del dataset. In fase di progettazione su Miro sono stati quindi individuati gli attributi essenziali per la caratterizzazione delle istanze (ad esempio identificativi, valori quantitativi, parametri o categorie testuali), successivamente formalizzati in Protégé come data properties, specificandone il tipo di dato (datatype) e gli eventuali vincoli sul valore.

Distinzione fra object and data properties e dataset

- Le **object properties** hanno lo scopo di rappresentare formalmente le relazioni tra entità del dominio. Siccome in OWL/DL le relazioni sono una parte fondamentale della conoscenza, esse permettono non solo di “collegare” individui tra loro, ma anche di introdurre vincoli logici utili al reasoning: ad esempio stabilire che un certo tipo di individuo deve essere collegato ad altri individui tramite una relazione specifica, imporre cardinalità, oppure determinare automaticamente appartenenze a classi.

- Le **data properties**, invece, hanno lo scopo di descrivere gli individui tramite attributi misurabili o descrittivi espressi con valori letterali (numeri, stringhe, date). Servono quindi a completare la rappresentazione: molte informazioni reali non sono relazioni verso altri individui, ma caratteristiche come un valore numerico, un identificativo o un parametro.
- Infine, il **dataset** (cioè la parte ABox popolata con individui) ha lo scopo di rendere l'ontologia "operativa": senza individui l'ontologia resta uno schema concettuale; con gli individui, invece, il reasoner può applicare le definizioni della TBox e i vincoli delle proprietà per **dedurre informazione implicita**, verificare coerenza e rispondere a interrogazioni. In altre parole, è il dataset che consente di passare da "questa è la struttura del dominio" a "questa è una situazione concreta nel dominio su cui posso fare reasoning e query".

Nel nostro specifico caso si è studiato un approccio semplificativo che escludesse l'uso dei data_set per evitare il mapping. In seguito, si riporta la gerarchia delle Object Properties e delle data Properties.

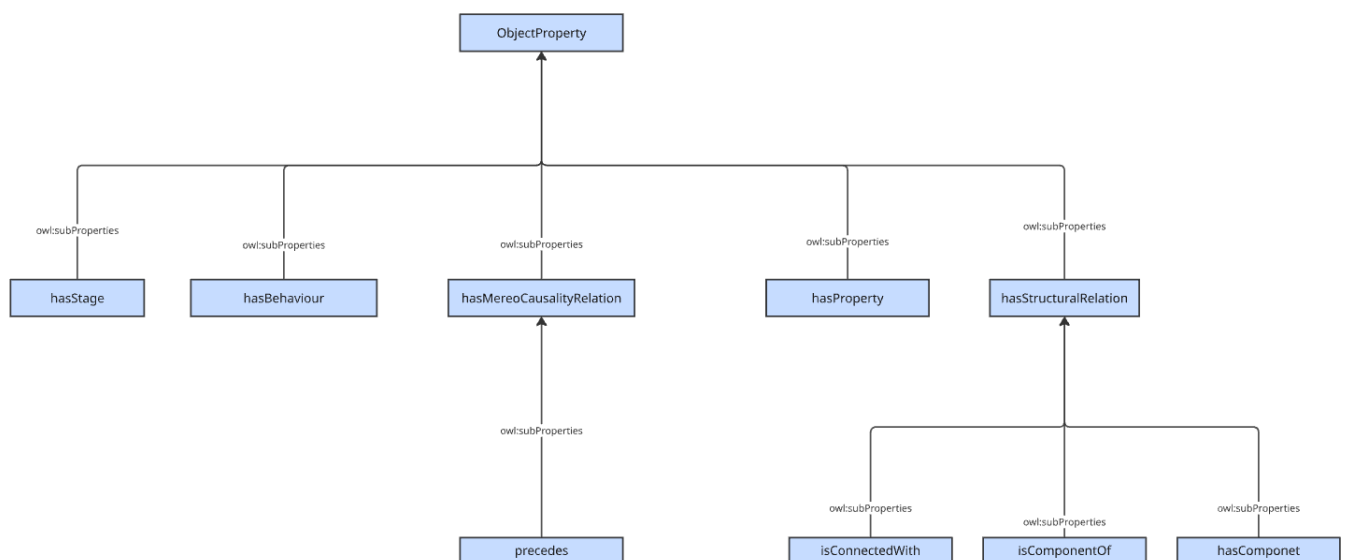


Figura 2: Object Properties Hierarchy

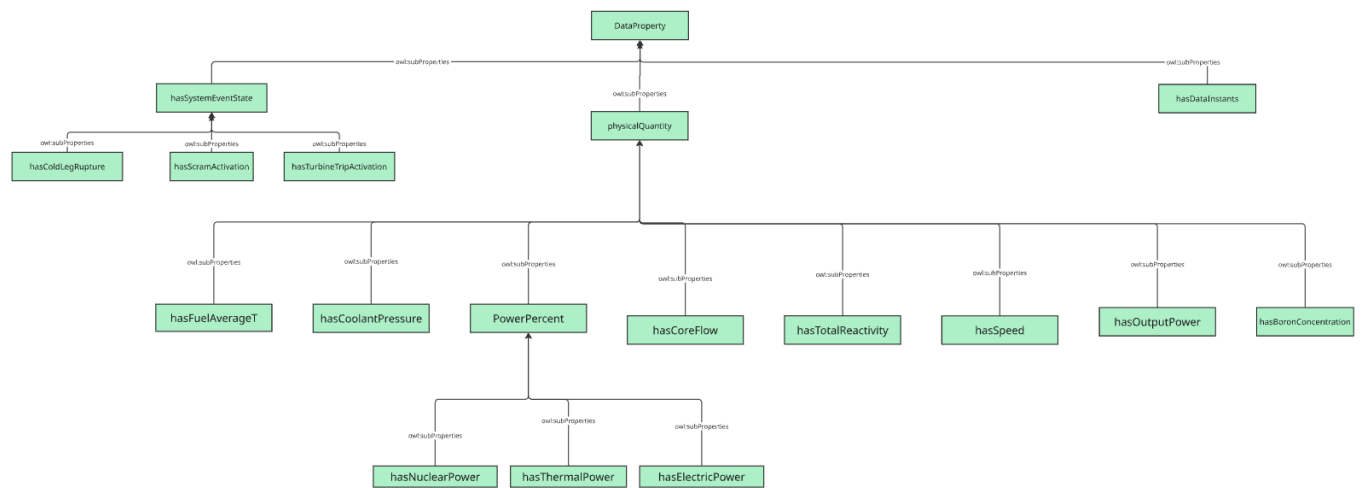


Figura 3: Data Properties Hierarchy

Cap3: Approccio su Protégé

Come accennato in precedenza, nel caso di studio considerato si è scelto di adottare un approccio semplificativo che escludesse l'uso esplicito di *dataset* come entità ontologiche autonome, al fine di evitare la fase di *mapping* verso risorse esterne. In questa scelta si è consapevolmente rinunciato a una modellazione più articolata della simulazione, concentrando invece l'attenzione sui dati prodotti e sul loro significato semantico.

In particolare, questo approccio ha comportato la perdita della rappresentazione della simulazione come *Functional Icon* secondo la terminologia di EMMO, nonché l'eliminazione di classi dedicate quali *DataSet* e *SimulationTime*. Tuttavia, tale semplificazione è risultata accettabile e coerente con gli obiettivi del progetto, che non mirano alla costruzione di un'ontologia nucleare completa, bensì alla comprensione dei principi fondamentali del Semantic Web e della modellazione ontologica.

Dal punto di vista implementativo, si è scelto di associare direttamente agli individui della A-box le informazioni numeriche prodotte dalle simulazioni. In particolare, per ogni individuo appartenente alla simulazione 1 o alla simulazione 2 (distinte rispettivamente tramite le etichette *sim1* e *sim2*), sono state definite in Protégé:

- una stringa di istanti temporali tramite una data property dedicata (es. *hasDataInstants*);
- una stringa di valori numerici per ciascuna specifica *physical quantity* o variabile booleana di interesse (es. *hasOutputPower*, *hasNeutronPower*, *hasScramActivation*).

Ogni individuo possiede esclusivamente le data properties pertinenti alle grandezze fisiche o agli eventi che lo caratterizzano, evitando ridondanze e mantenendo una chiara corrispondenza semantica tra oggetto e dato. Questo approccio è risultato praticabile poiché il numero di classi coinvolte è relativamente limitato e il confronto riguarda unicamente due simulazioni.

In una fase iniziale del lavoro era stata ipotizzata una modellazione più ricca, con un maggior numero di classi e una rappresentazione esplicita dei dataset e delle simulazioni. Tuttavia, con l'avanzare dell'attività si è preferito privilegiare una struttura ontologica più compatta, in cui ogni classe fosse effettivamente popolata da dati reali, logicamente coerente e semanticamente ben definita. Questa scelta rispecchia l'obiettivo didattico del progetto, orientato alla comprensione dei meccanismi di base della modellazione ontologica piuttosto che alla completezza del dominio nucleare.

A titolo di esempio, si riporta il contenuto visualizzato selezionando l'individuo *AP600_1* nella sezione *Individuals* di Protégé.



Figura 4: Esempio di Individuo (*AP600_1*)

Come si osserva, l'utilizzo delle data properties consente di eliminare la dipendenza da tabelle esterne, rendendo disponibili tutte le informazioni rilevanti direttamente all'interno dell'ontologia. Inoltre, per evitare qualsiasi ambiguità interpretativa, a ciascuna data property è stata associata un'annotazione che specifica esplicitamente l'unità di misura della grandezza rappresentata.

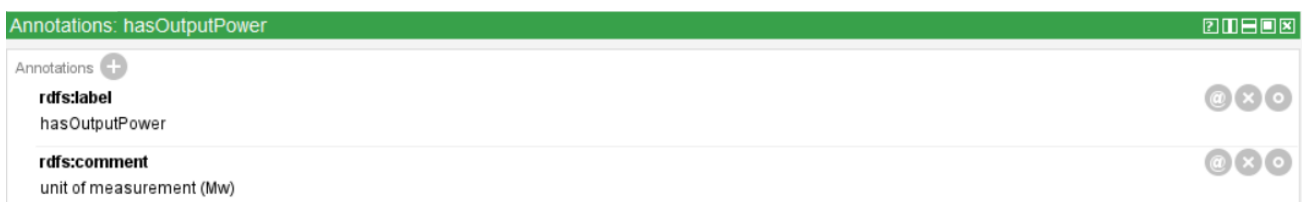


Figura 5: Esempio di data Property (*hasOutputPower*)

Per chiarezza, nella sezione degli individui è stata inoltre inserita un'annotazione testuale che indica che tutti gli individui con etichetta "1" fanno riferimento alla simulazione 1, mentre quelli con etichetta "2" alla simulazione 2.

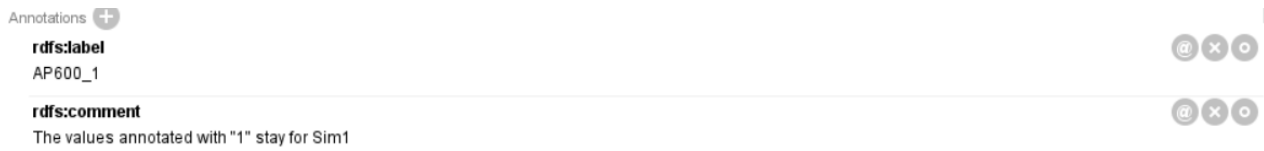


Figura 6: Annotation *Sim1*

Un ulteriore aspetto rilevante riguarda la modellazione di eventi discreti, quali lo SCRAM, il *Turbine Trip* e la *Cold Leg Rupture*. Tali fenomeni sono stati rappresentati mediante variabili booleane, che restituiscono per ciascun istante temporale lo stato dell'evento considerato. In questo modo, eventi di natura discreta vengono integrati coerentemente nella stessa struttura temporale delle grandezze continue.

Ad esempio, selezionando l'individuo *SCRAM1* è possibile osservare direttamente la sequenza booleana associata agli istanti temporali della simulazione 1.



Figura 7: Variabile Boolean Scram confronto con Istanti di Tempo (Sim1)

Attraverso questo approccio si rinuncia alla simulazione intesa come *Functional Icon* in senso EMMO, ma si elimina anche la necessità di affrontare il mapping verso strumenti esterni come OntoRefine. Al contempo, si ottiene una maggiore evidenza dei dati e la possibilità di effettuare direttamente operazioni di querying su GraphDB, sfruttando una A-box completamente integrata in Protégé.

In questo contesto, il lavoro preliminare svolto tramite mappe concettuali in Miro si è rivelato essenziale: senza una fase di concettualizzazione chiara e condivisa, la costruzione di una A-box coerente, compatta e semanticamente consistente non sarebbe stata possibile.

Cap4: Reference su GraphDB e querying

Volendo fare un breve recap del lavoro svolto su GraphDB, partiamo anzitutto col darne la definizione. GraphDB è un database semantico (triplestore) grazie al quale è possibile memorizzare i file RDF (*Resource Description Framework*), interrogare i dati con SPARQL (linguaggio standard d'interrogazione per file RDF), ragionare sugli stessi ed infine visualizzare grafici e relazioni complesse tra questi. Un file RDF, inoltre, può essere scritto in uno dei formati che seguono:

- .ttl (Turtle)
- .rdf (RDF/XML)
- .nt (N-Triples)
- .jsonld (JSON-LD)

La struttura a grafo che caratterizza GraphDB, consente la creazione delle cosiddette query semantiche, mediante utilizzo di nodi ed archi; da ciò, deriva proprio il nome di struttura a grafo (archi, relazione). Quest'ultimo, in particolare, rappresenta il collegamento tra i vari elementi di dati nell'archivio e i diversi nodi/archi, i quali a loro volta rappresentano le relazioni tra i nodi. Trattandosi dunque di un database a grafo, GraphDB considera le relazioni tra i dati come una "priorità" e l'interrogazione delle relazioni è alquanto veloce essendo queste continuamente memorizzate all'interno dello stesso database. Queste, inoltre, possono essere visualizzate in maniera intuitiva mediante questo particolare tipo di database, e questo è il motivo per cui tools come questo sono particolarmente utili quando si trattano dati fortemente interconnessi.

A differenza di altri database tradizionali, sulla base di quanto appena detto, è facile comprendere che GraphDB non salva tabelle bensì triple RDF, nonché file con una struttura del tipo:

Soggetto – Predicato – Oggetto

Il fatto che GraphDB lavori con questa particolare tipologia di file, implica che i dati siano rappresentati come grafi e che ogni dato/relazione sia identificata da uno specifico URI (*Uniform Resource Identifier*). Questo, fa sì che ognuno di questi sia univoco, interpolabile e collegabile ad un altro. I campi di applicazione cui è destinato GraphDB sono alquanto vasti, a partire dal Web Semantico, sino al campo del Data integration o dell'Industria 4.0, e a supporto di questa innumerevole serie di applicazioni, sicuramente il fatto che grazie a questo database è possibile trattare i dati con significato, vale a dire che questi non vengono considerati come numeri, ma come relazioni e/o concetti. Chiaramente, ciò consente di realizzarne un collegamento e quindi di unire diverse fonti senza modificare lo schema di partenza, così da poter dedurre anche informazioni nuove. Nel contesto del nostro studio, al fine di poter raggiungere l'obiettivo finale, GraphDB è stato utilizzato per poter ottenere le query a partire dal file elaborato tramite Protegè. In particolare, il file RDF da noi usato è stato scritto in formato .ttl (Turtle), il quale è stato prima importato su Github e, dopo averne copiato l'OWL è stato usato per creare il repository su GraphDB.

Successivamente, è stato possibile realizzare le query SPARQL, esplorare le varie classi/individui/proprietà ed infine verificare che tutti i dati fossero corretti (oltre che ragionevoli).

L'uso delle query SPARQL è dovuto al fatto che queste consentono di definire dei namespace ben precisi, usare delle variabili, specificare i pattern di triple sopra presentati, filtrare i risultati ed infine restituire i dati. La loro creazione prevede di seguire degli step ben precisi, di seguito enunciati:

- 1) Si parte con l'identificare tutte le classi e proprietà a partire dal file .ttl di partenza
- 2) Si scrive il pattern RDF (ricordando la struttura "triple" dei dati)
- 3) Si costruisce infine la query vera e propria

Bisogna inoltre aver chiaro il fatto che le query SPARQL che possono essere implementate su GraphDB possono essere di vario tipo, la cui scelta dipende chiaramente dall'obiettivo cui si desidera pervenire, ma qualunque sia quella realizzata, la struttura presenta per ciascuna delle parti in comune, nonché i PREFIX, utili per l'abbreviazione degli URI, il "WHERE" che contiene i pattern di triple e le VARIABILI (introdotte mediante il simbolo "?". Volendo essere adesso più specifici, per arrivare alla struttura finale della query per prima cosa è necessaria un'analisi dell'ontologia di partenza, la quale richiede di identificare i vari namespace, individuare le classi, le proprietà e i label. Successivamente bisogna che sia chiaro l'obiettivo cui si vuole arrivare (ad esempio: ottenere le classi, recuperare le proprietà, esplorare le relazioni), così da partire con la scrittura dei pattern RDF.

Si riportano le parti di codice usate per la costruzione delle query per il file ap600_2.ttl da cui si è partiti:

1) Prefissi usati:

```
PREFIX : <http://www.ap600.com/ontologies/ap600.owl/>  
PREFIX owl: <http://www.w3.org/2002/07/owl#>  
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

2) Query 1: presentazione di tutte le classi dell'ontologia

```
SELECT ?class ?label
WHERE {
  ?class a owl:Class .
  OPTIONAL { ?class rdfs:label ?label }
}
ORDER BY ?class
```

3) Query 2: classi + label

```
SELECT ?property ?type ?label
WHERE {
  ?property a ?type .
  FILTER(?type IN (owl:ObjectProperty,
  owl:DatatypeProperty))
  OPTIONAL { ?property rdfs:label ?label }
}
ORDER BY ?property
```

4) Query 3: presentazione di tutte le proprietà

```
SELECT DISTINCT ?entity ?label
WHERE {
  ?entity rdfs:label ?label .
}
ORDER BY ?label
```

5) Query 4: proprietà + label

```
SELECT ?property ?domain ?range
WHERE {
  ?property rdfs:domain ?domain .
  OPTIONAL { ?property rdfs:range ?range }
}
ORDER BY ?property
```

6) Query 5: presentazione dei label nel file

```
SELECT ?property
WHERE {
    ?property a owl:ObjectProperty .
}
ORDER BY ?property
```

7) Query 6: dominio e range delle proprietà

```
SELECT ?property
WHERE {
    ?property a owl:DatatypeProperty .
}
ORDER BY ?property
```

8) Query 7: gerarchia delle classi

```
SELECT ?class ?subclass
WHERE {
    ?subclass rdfs:subClassOf ?class .
}
ORDER BY ?class
```

9) Query 8: esplorazione totale

```
SELECT ?s ?p ?o
WHERE {
    ?s ?p ?o .
}
LIMIT 200
```

10) Query 9: serve per capire se i valori esistono davvero (trova tutte le datatype property, mostra i valori e si accerta che tutte le stringhe, date e numeri funzionino)

```
PREFIX : <http://www.ap600.com/ontolgies/ap600.owl/>

SELECT ?object ?property ?value
WHERE {
    ?object ?property ?value .
    FILTER(isLiteral(?value))
}
ORDER BY ?object ?property
```

11) Query 10: query per vedere tutte le datatype property per ciascuno degli oggetti

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX : <http://www.ap600.com/ontologies/ap600.owl/>

SELECT ?object ?property ?value
WHERE {
    ?object ?property ?value .
    ?property a owl:DatatypeProperty .
}
ORDER BY ?object ?property
```

Grazie alle query strutturate, in particolare l'ultima presentata, oltre alla object property, ha fatto sì che si potessero interrogare anche le datatype property dell'ontologia. Tramite SPARQL, inoltre, si è potuto recuperare per ogni istanza i valori letterali, quali stringhe e date che associassero a ciascuno oggetto una ben precisa proprietà. Si riporta l'output ottenuto da quest'ultima query, fondamentale ai fini dello studio condotto in quanto ha permesso di verificare la validità delle stringhe riportate nell'ontologia sviluppata.

	object	property	value
1	:AP600_1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
2	:AP600_1	:hasGeneratorOutputPower	"[623; 625; 629; 629; 563; 0; 0; 0]"
3	:AP600_1	:hasPhysicalQuantity	"[623; 625; 629; 629; 563; 0; 0; 0]"
4	:AP600_2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
5	:AP600_2	:hasGeneratorOutputPower	"[440; 440; 442; 439; 439; 415; 28; 0; 0]"
6	:AP600_2	:hasPhysicalQuantity	"[440; 440; 442; 439; 439; 415; 28; 0; 0]"
7	:Core1	:PowerPercentage	"[100; 100; 100; 7; 0; 0; 0; 0]"
8	:Core1	:PowerPercentage	"[100; 100; 100; 87; 8; 5; 3; 3]"
9	:Core1	:hasCoreFlow	"[9206; 9053; 8690; 8914; 8959; 6510; 162; 229]"
10	:Core1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
11	:Core1	:hasFuelAverageT	"[484; 484; 485; 453; 292; 268; 238; 264]"
12	:Core1	:hasNeutronPower	"[100; 100; 100; 7; 0; 0; 0; 0]"
13	:Core1	:hasPhysicalQuantity	"[9206; 9053; 8690; 8914; 8959; 6510; 162; 229]"
14	:Core1	:hasPhysicalQuantity	"[484; 484; 485; 453; 292; 268; 238; 264]"
15	:Core1	:hasPhysicalQuantity	"[100; 100; 100; 7; 0; 0; 0; 0]"
16	:Core1	:hasPhysicalQuantity	"[100; 100; 100; 87; 8; 5; 3; 3]"
17	:Core1	:hasPhysicalQuantity	"[0; 0; 0; -119; -197; -192; -187; -191]"
18	:Core1	:hasThermalPower	"[100; 100; 100; 87; 8; 5; 3; 3]"

30	:Core2	:hasPhysicalQuantity	"[0; 0; 0; 0; 0; -195; -194; -181; -178]"
31	:Core2	:hasThermalPower	"[70; 70; 70; 69; 70; 14; 5; 3; 2]"
32	:Core2	:hasTotalReactivity	"[0; 0; 0; 0; 0; -195; -194; -181; -178]"
33	:LOCA1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
34	:LOCA2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
35	:Loop1	:hasBoronConcentration	"[213; 213; 213; 213; 212; 210; 199; 188]"
36	:Loop1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
37	:Loop1	:hasPhysicalQuantity	"[213; 213; 213; 213; 212; 210; 199; 188]"
38	:Loop2	:hasBoronConcentration	"[209; 209; 209; 209; 209; 208; 203; 150; 95]"
39	:Loop2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
40	:Loop2	:hasPhysicalQuantity	"[209; 209; 209; 209; 209; 208; 203; 150; 95]"
41	:RCS1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
42	:RCS2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
43	:SCRAM1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
44	:SCRAM1	:hasScramActivation	"[0; 0; 0; 1; 1; 1; 1; 1]"
45	:SCRAM1	:hasSystemEventState	"[0; 0; 0; 1; 1; 1; 1; 1]"
46	:SCRAM2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
47	:SCRAM2	:hasScramActivation	"[0; 0; 0; 0; 0; 1; 1; 1]"
48	:SCRAM2	:hasSystemEventState	"[0; 0; 0; 0; 0; 1; 1; 1]"
52	:Turbine1	:hasPhysicalQuantity	"[1800; 1800; 1800; 1800; 1800; 1780; 5; 5]"
53	:Turbine1	:hasSpeed	"[1800; 1800; 1800; 1800; 1800; 1780; 5; 5]"
54	:Turbine2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
55	:Turbine2	:hasPhysicalQuantity	"[1800; 1800; 1800; 1800; 1800; 1800; 1800; 5; 5]"
56	:Turbine2	:hasSpeed	"[1800; 1800; 1800; 1800; 1800; 1800; 1800; 5; 5]"
57	:TurbineTrip1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
58	:TurbineTrip1	:hasSystemEventState	"[0; 0; 0; 0; 0; 1; 1; 1]"
59	:TurbineTrip1	:hasTripActivation	"[0; 0; 0; 0; 0; 1; 1; 1]"
60	:TurbineTrip2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
61	:TurbineTrip2	:hasSystemEventState	"[0; 0; 0; 0; 0; 0; 1; 1]"
62	:TurbineTrip2	:hasTripActivation	"[0; 0; 0; 0; 0; 0; 1; 1]"
63	:Vessel1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
64	:Vessel2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
65	:coldLeg1	:hasColdLegRupture	"[0; 1; 1; 1; 1; 1; 1; 1]"
66	:coldLeg1	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
67	:coldLeg1	:hasSystemEventState	"[0; 1; 1; 1; 1; 1; 1; 1]"
68	:coldLeg2	:hasColdLegRupture	"[0; 1; 1; 1; 1; 1; 1; 1]"
69	:coldLeg2	:hasDataInstants	"[0; 10; 20; 30; 40; 65; 185; 385]"
70	:coldLeg2	:hasSystemEventState	"[0; 1; 1; 1; 1; 1; 1; 1]"
71	:electricGenerator1	:PowerPercentage	"[100; 101; 101; 101; 91; 0; 0; 0]"

Figura 8: Esempio di risultato

Volendo cingerci ad una conclusione, possiamo ribadire che nel caso studio in esame l'uso di GraphDB, in qualità di database semantico, è stato fondamentale al fine di memorizzare ed interrogare l'ontologia OWL relativa al file dei dati di simulazione per il reattore AP600. Grazie inoltre al linguaggio SPARQL, implementato nello stesso database, è stata possibile la gestione dei dati in modo semantico, così da poter favorire l'esplorazione strutturata delle varie classi, proprietà e relazioni, arrivando a dimostrare come tecnologie del mondo semantico quali GraphDB consentano una gestione dei dati più intelligente ed espressiva, rispetto quella cui si perverrebbe usando database tradizionali.

Conclusioni:

Il lavoro svolto ha permesso di accompagnare l'intero ciclo di trasformazione dell'informazione, partendo da dati numerici grezzi generati da simulazioni ingegneristiche fino a una loro rappresentazione semantica interrogabile. In una fase iniziale, i risultati delle simulazioni effettuate con il software certificato Passive PWR erano organizzati esclusivamente sotto forma di tabelle Excel, efficaci per l'analisi numerica ma intrinsecamente limitate dal punto di vista semantico, poiché prive di un'esplicita formalizzazione dei concetti, delle relazioni e degli eventi che caratterizzano il transitorio simulato.

Attraverso una fase preliminare di modellazione concettuale su Miro, è stato possibile chiarire la struttura del dominio di interesse, distinguendo tra TBox e ABox e definendo in modo consapevole le classi, le proprietà e le relazioni rilevanti. Questa attività ha rappresentato un passaggio fondamentale per garantire coerenza concettuale, evitando ambiguità e sovrapposizioni semantiche nella successiva implementazione formale.

La realizzazione dell'ontologia in OWL tramite Protégé ha consentito di strutturare i dati di simulazione come informazioni semanticamente arricchite. In particolare, l'adozione di un approccio basato su data properties contenenti stringhe di valori temporali ha permesso di superare la necessità di un mapping esterno dei dataset, mantenendo l'intera ABox direttamente all'interno dell'ontologia. In questo modo, le grandezze fisiche e gli eventi discreti di interesse operativo e di sicurezza sono stati formalizzati come attributi espliciti degli individui, ciascuno dotato di significato, unità di misura e contesto ben definito.

Infine, l'importazione dell'ontologia in GraphDB ha reso possibile l'interrogazione diretta del sistema tramite query semantiche, evidenziando il valore aggiunto dell'approccio ontologico rispetto alla semplice gestione tabellare dei dati. I risultati delle simulazioni non sono più interpretati come valori isolati, ma come informazioni coerenti, strutturate e logicamente connesse, aprendo la strada a confronti tra scenari, verifiche di coerenza e analisi più avanzate. Nel complesso, il progetto ha raggiunto l'obiettivo di fornire una comprensione concreta e operativa delle basi del Semantic Web, dimostrando come strumenti ontologici possano supportare efficacemente l'organizzazione e l'interpretazione dei dati di simulazione in ambito ingegneristico.

Bibliografia:

Il presente lavoro si basa su materiali didattici forniti durante il corso e su software utilizzati a scopo esclusivamente educativo.

1. **European Materials Modelling Ontology (EMMO):** European Materials Modelling Ontology (EMMO).
EMMO – A multidisciplinary top-level ontology for materials modelling.
Available online: <https://emmo-repo.github.io>
Accessed: 2025.
2. **Emanuele Ghedini :** *Lecture notes and slides on Semantic Web and Ontologies.*
University of Bologna, academic year 2024–2025.
(Materiale didattico fornito durante il corso).
3. **International Atomic Energy Agency (IAEA):** International Atomic Energy Agency.
Passive PWR Simulator (Passive_PWR).
IAEA educational and training software for nuclear reactor simulations.
4. **Passive PWR:** Certified nuclear reactor simulation software used for educational purposes, developed under the framework of IAEA nuclear training tools.
5. **Miro:** *Online collaborative whiteboard platform.*
Used for conceptual modeling and preliminary ontology design.
<https://miro.com>
6. **Protégé:** Stanford Center for Biomedical Informatics Research.
Protégé Ontology Editor and Knowledge Acquisition System.
<https://protege.stanford.edu>
7. **GraphDB:** *GraphDB – Semantic Graph Database.*
RDF triplestore used for ontology storage and SPARQL querying.
<https://graphdb.ontotext.com>
8. **Graph database.** In Wikipedia. Wikimedia Foundation, from
https://en.wikipedia.org/wiki/Graph_database

9. **Definizioni di ABox e di TBox e loro differenze:** 'THE DESCRIPTION LOGIC HANDBOOK: Theory, implementation, and applications' by Franz Baader, Deborah L. McGuinne, Daniele Nardi and Peter F. Patel-Schneider